

Flag : `picoCTF{puzzl3d_m3tadata_f0und!_c999e2a4}`

Résumé de la faille

Un PDF affiche un texte "leurre" qui prétend qu'il n'y a rien à trouver ("No flag here. Nice try though!"). Le flag n'est ni dans le texte visible ni dans le texte caché du contenu — il est planqué dans les **métadonnées** du fichier (le champ Author), encodé en base64.

Concept : le contenu visible d'un fichier n'est qu'une partie de ce qu'il contient. Les métadonnées sont une couche invisible à l'utilisateur normal, souvent négligée — et donc un bon endroit pour cacher (ou faire fuiter) de l'information.

Méthodo — les étapes

1. Lire l'énoncé attentivement - Texte volontairement trompeur ("just random text", "wrong place", "the answer might not be here after all"). Piste : chercher dans ce qui n'est PAS le texte visible du PDF.

2. Localiser le fichier sur Kali - Vérifier l'emplacement avec `pwd / ls -la` - Si le fichier est sur le Bureau : utiliser le chemin `Bureau/confidential.pdf` (ou `~/Desktop/...` selon la langue du système)

3. Premier réflexe : `strings`

```
strings Bureau/confidential.pdf | grep -i pico
strings Bureau/confidential.pdf | grep -i flag
```

- Résultat : `/Flags 32` (plusieurs fois) → **faux positif**, c'est une propriété technique interne du format PDF (F majuscule), pas notre flag.
- Rien de concluant → le flag n'est pas en texte lisible dans le contenu.

4. Deuxième réflexe : `exiftool` (les métadonnées)

```
exiftool Bureau/confidential.pdf
```

- Résultat clé, dans le champ **Author** : `cG1jb0NURntwdXp6bDNkX20zdGFkYXRhX2YwdW5kIV9jOTk5ZTJhNH0=`
- Une chaîne bizarre à cet endroit (au lieu d'un vrai nom) = suspect.

5. Reconnaître l'encodage - La chaîne finit par `=` (padding) et a une longueur cohérente → signature classique du **base64**.

6. Décoder (CyberChef) - Coller la chaîne dans CyberChef → opération **From Base64** - Résultat : `picoCTF{puzzl3d_m3tadata_f0und!_c999e2a4}` P

Réflexe forensics — la triade de départ

Sur n'importe quel fichier inconnu à analyser :

- `file fichier` → quel type de fichier c'est vraiment
- `strings fichier` → chaînes de texte lisibles cachées dedans
- `exiftool fichier` → métadonnées (auteur, dates, logiciel, GPS...)

Ces trois commandes couvrent une grande partie des cas simples, avant de sortir l'artillerie plus lourde (`binwalk` , `pdf-parser` , `peepdf` ...).

Outils utilisés

Outil	Rôle
<code>strings</code>	Extraire les chaînes de texte lisibles d'un binaire
<code>exiftool</code>	Lire les métadonnées d'un fichier (auteur, dates, logiciel...)
CyberChef	Décoder le base64 trouvé

Réflexes à retenir

- Un texte qui dit "il n'y a rien ici" dans un CTF est presque toujours un mensonge/indice détourné — chercher ce qui n'est pas affiché.
- `strings` + `exiftool` sont les deux premiers réflexes sur un fichier suspect, avant les outils plus poussés.
- Se méfier des faux positifs qui ressemblent au flag de loin (`/Flags 32` ≠ `picoCTF{...}`) — vérifier le format exact attendu.
- Une chaîne bizarre dans un champ censé contenir du texte normal (auteur, titre...) = suspect → tester un décodage (base64 en premier réflexe, vu le `=` et la longueur).

Dans la vraie vie : pourquoi les métadonnées comptent

Les métadonnées fuient souvent des informations sensibles sans que l'utilisateur s'en rende compte : - Photos avec coordonnées **GPS** exactes (localisation) - PDF/Word avec le **vrai nom de l'auteur** dans un document censé être anonyme - Historique de modification, nom de l'ordinateur, logiciel utilisé

Réflexe pro / vie privée : nettoyer les métadonnées avant de partager un fichier publiquement :

```
exiftool -all= fichier.pdf
```

Utilisé en **OSINT** (extraire un maximum d'infos d'un fichier public) et en **hygiène numérique** (éviter de fuiter des infos par erreur).

